

RedNorte

Servicio Público de Salud — Plataforma Inteligente para la Gestión de Listas de Espera Hospitalarias

Autor: Sebastián Godoy

Fecha: Junio 2026

Institución: Instituto Profesional DuocUC

Asignatura: DSY1106 — Desarrollo Fullstack III

1. Diseño de Arquitectura y Patrones

1.1. Introducción y Contexto

El Servicio Público de Salud RedNorte administra una red de hospitales, centros de atención primaria y clínicas especializadas en el norte del país. Uno de sus principales desafíos es la gestión eficiente de las listas de espera para consultas médicas, procedimientos y cirugías. Este proyecto consiste en una plataforma digital basada en microservicios que centraliza la gestión de listas de espera, optimiza la asignación de horas médicas y proporciona visibilidad a los pacientes sobre el estado de sus solicitudes.

1.2. Arquitectura Propuesta

La arquitectura se compone de cuatro capas: presentación, API Gateway, lógica de negocio y persistencia. Los componentes se ejecutan como contenedores Docker, orquestados con Docker Compose para desarrollo y Kubernetes para producción.



Frontend: Astro 4 con Preact 10 para componentes interactivos, Nanostores para estado global y Tailwind CSS 3. En desarrollo corre en el puerto 4321 con Bun. En Kubernetes se sirve mediante NGINX en el puerto 80.

API Gateway: KrakenD reemplaza al BFF tradicional. Toda la lógica de enrutamiento se define en un archivo JSON. Expone 8 endpoints, maneja CORS y propaga el header Authorization a los microservicios que lo requieren.

Microservicios: Tres servicios independientes desarrollados con Hono 4 sobre Bun 1.3.14. Bun incluye test runner, gestor de paquetes y compilador integrados.

- **ms-users** (:3001) — registro, login, JWT. Hash Argon2 con Bun.password.
- **ms-waitlist** (:3000) — CRUD de lista de espera con prioridades (1-4) y estados. Valida JWT con @hono/jwt. Publica eventos en RabbitMQ.
- **ms-notifications** (:3002) — consume RabbitMQ, persiste y envía notificaciones vía WebSocket nativo de Bun.

Persistencia: Cada microservicio tiene su propia base de datos PostgreSQL 16 independiente (Database per Service). Las migraciones se ejecutan al iniciar el contenedor. En Kubernetes se usan StatefulSets con volúmenes persistentes y Jobs de migración independientes.

1.3. Autenticación

1. Registro: POST /api/auth/users — valida email único, hash con Argon2, guarda en users_db.
2. Login: POST /api/auth/users/login — verifica hash, firma JWT con jsonwebtoken.
3. JWT: { id, email, role_id } con expiración de 2 horas.

4. Validación: ms-waitlist usa @hono/jwt middleware en todas sus rutas.
5. Ruta /mine: extrae userId del JWT en lugar de recibirlo por query param.

1.4. Patrones de Diseño

- **Repository Pattern:** encapsula consultas SQL en repositorios.
- **Service Layer:** lógica de negocio en servicios, no en controladores.
- **Controller:** orquesta request y response únicamente.
- **Centralized Error Handler:** AppError con statusCode capturado por app.onError().
- **Observer:** ms-waitlist publica eventos en RabbitMQ; ms-notifications consume y envía WebSocket.
- **Connection Manager:** Map<userId, WebSocket> en ms-notifications.

2. Especificaciones Técnicas

Componente	Tecnología	Versión
Runtime backend	Bun	1.3.14
Framework backend	Hono	4.12.x
API Gateway	KrakenD	2.6.3 / 2.13.7
Lenguaje	TypeScript	5.x
Frontend framework	Astro	4.x
Componentes	Preact	10.x
Estado global	Nanostores	última
Estilos	Tailwind CSS	3.x
Motor BD	PostgreSQL (alpine)	16
Driver BD	postgres (nativo)	3.x
Mensajería	RabbitMQ	3-management
Hash de contraseñas	Bun.password (Argon2)	nativo
Testing backend	Bun test	nativo
Testing frontend	Vitest + happy-dom	4.x
Cobertura	@vitest/coverage-v8	última
Orquestación desarrollo	Docker Compose	última
Orquestación producción	Kubernetes + Kustomize	1.x
Servidor frontend (K8s)	NGINX alpine	última

3. Integración con Kubernetes

El sistema incluye soporte completo para despliegue en Kubernetes como alternativa a Docker Compose, utilizando Kustomize para la gestión declarativa de recursos.

3.1. Estructura

```
k8s/
├── kustomization.yaml # Recursos raíz
├── namespace.yaml    # Namespace "rednorte"
├── secrets.yaml       # Credenciales y JWT
├── ingress.yaml       # Ingress con soporte WebSocket
├── postgres/         # StatefulSets (users, waitlist, notifications)
├── rabbitmq/         # StatefulSet
├── jobs/             # Migraciones (batch Jobs)
├── ms-users/         # Deployment + Service
├── ms-waitlist/       # Deployment + Service
├── ms-notifications/ # Deployment + Service
├── krakend/          # ConfigMap + Deployment + Service
└── frontend/         # Deployment + Service (NGINX)
```

3.2. Componentes

Recurso	Tipo	Función
Namespace	v1/Namespace	Aisla recursos en rednorte
Secrets	v1/Secret	POSTGRES_USER, POSTGRES_PASSWORD, JWT_SECRET, RabbitMQ credenciales
PostgreSQL (x3)	StatefulSet	Bases independientes con volúmenes persistentes
RabbitMQ	StatefulSet	Bus de eventos
Migration Jobs	batch/v1/Job	Ejecutan migrate.ts contra cada BD
Microservicios	Deployment	ms-users (:3001), ms-waitlist (:3000), ms-notifications (:3002)
KrakenD	Deployment + ConfigMap	Configuración inyectada desde ConfigMap
Frontend	Deployment	NGINX sirviendo el build de Astro (:80)
Ingress	networking.k8s.io/Ingress	/api → KrakenD, /ws → ms-notifications, / → frontend

3.3. Flujo de Despliegue

1. Construir imágenes Docker: `docker build -t rednorte/ms-users:latest ./backend/ms-users` (similar para cada servicio y frontend).
2. Aplicar Kustomize: `kubectl apply -k k8s/`.
3. Esperar que los pods estén listos: `kubectl wait --for=condition=ready pods --all -n rednorte --timeout=180s`.
4. Acceder vía port-forward: `kubectl port-forward -n rednorte service/frontend 4321:80`.

El script `scripts/deploy-k8s.ps1` automatiza todo el proceso.

3.4. Diferencias con Docker Compose

Aspecto	Docker Compose	Kubernetes
Frontend	Bun dev server (:4321)	NGINX sirviendo build (:80)
Config KrakenD	Volumen montado desde host	ConfigMap en el pod
Bases de datos	Volúmenes Docker locales	StatefulSets con PVC

Migraciones	En CMD del contenedor	Jobs independientes
Escalabilidad	Manual	Auto-escalado horizontal
Health checks	Healthcheck en Docker	Liveness + Readiness probes
Ingreso tráfico	localhost	Ingress NGINX con dominio
WebSocket	Conexión directa a ms-notifications	Ingress con anotaciones WebSocket

4. Entregables

Tipo	Ubicación	Descripción
Código fuente	github.com/GoguX76/rednorte	Monorepo completo
ms-users	backend/ms-users/	Autenticación y usuarios
ms-waitlist	backend/ms-waitlist/	Lista de espera + RabbitMQ
ms-notifications	backend/ms-notifications/	Notificaciones WebSocket
API Gateway	backend/bff/krakend.json	Config KrakenD (8 endpoints)
Frontend	frontend/	Astro + Preact + Tailwind
Kubernetes	k8s/	Manifiestos completos + Kustomize
Script K8s	scripts/deploy-k8s.ps1	Despliegue automatizado
Diagrama C1	docs/diagrams/rednorte-c1.png	Contexto del sistema
Diagrama C2	docs/diagrams/rednorte-c2.png	Contenedores
Diagrama C3	docs/diagrams/rednorte-c3.html	Componentes internos
Persistencia	docs/data-persistence.pdf	Documentación de BD
Pruebas	docs/unit-test-report.pdf	94 tests, ≥90% cobertura
API	docs/postman-collection.json	8 endpoints
READMEs	Cada componente	Instrucciones + ficha técnica

5. Conclusión

La arquitectura implementada para RedNorte ofrece una solución desacoplada basada en microservicios. KrakenD como API Gateway elimina la necesidad de un BFF tradicional. Los tres microservicios están aislados con bases de datos independientes, siguiendo el principio de responsabilidad única.

RabbitMQ permite la comunicación asíncrona entre ms-waitlist y ms-notifications, garantizando la entrega de notificaciones incluso ante caídas temporales. Las notificaciones en tiempo real llegan al frontend mediante WebSocket nativo de Bun, con reconexión automática.

El frontend con Astro y Preact ofrece carga mínima al enviar cero JavaScript por defecto. La cobertura de pruebas unitarias supera el 90% en los cuatro componentes, con 94 pruebas automatizadas. Los patrones Repository, Service Layer, Controller y Centralized Error Handler mantienen el código mantenible y testeable.

El sistema soporta dos modos de despliegue: Docker Compose para desarrollo y presentaciones, y Kubernetes con Kustomize para producción, incluyendo NGINX para el frontend, StatefulSets para bases de datos, ConfigMaps para configuración, e Ingress para enrutamiento con soporte WebSocket.